

Assignment 06

Problem 0: Simple Functions

For this part you will be completing a series of short programming challenges that use various function concepts. Each challenge should be saved as its own file.

Challenge 1

Write two functions called 'maximum' and 'minimum' - these functions should accept two arguments and **return** the larger/smaller of the two supplied values. For the purpose of this program you can always assume that the arguments being supplied are numeric. **Describe your function using IPO notation.** Your program should work perfectly with the following code:

```
a = 5
b = 10
c = 15
d = 20

ans1 = maximum(a,b)
ans2 = maximum(a,c)
ans3 = maximum(a,d)
print (ans1,ans2,ans3) # 10 15 20

ans4 = minimum(d,c)
ans5 = minimum(d,b)
ans6 = minimum(d,a)
print (ans4,ans5,ans6) # 15 10 5

ans7 = maximum( maximum(a,b), maximum(c,d) )
print ("The biggest is:", ans7)
```

Challenge 2

Write a function called 'valid_date' which **accepts two arguments - a month and a day (both integers, will always be integers)**. Test to see if the date being described is valid or not. If the date is valid your function should **return a True value**, and if it is not it should return a **False** value. For the purpose of this program you can assume February has 28 days. **Describe your function using IPO notation.** Your program should work perfectly with the following code:

```
print (valid_date(99,1)) # False
print (valid_date(1,99)) # False
print (valid_date(99,99)) # False

print (valid_date(-99,1)) # False
print (valid_date(1,-99)) # False
print (valid_date(-99,-99)) # False

print (valid_date(9,25)) #True
print (valid_date(10,15)) # True
print (valid_date(11,31)) # False
print (valid_date(2,28)) # True
print (valid_date(2,29)) # False
```

Challenge 3

Write a function called 'simple_sort' which accepts three arguments. You can assume that they will always be integers. Sort these values in ascending order and return them. You may use the maximum/minimum functions if you'd like. Your function should work perfectly with the following lines of code:

```
a,b,c = simple_sort_version(10,20,30)
print (a,b,c) # 10 20 30

a,b,c = simple_sort_version(210,30,20)
print (a,b,c) # 10 20 30

a,b,c = simple_sort_version(30,20,10)
print (a,b,c) # 10 20 30

a,b,c = simple_sort_version(30,20,20)
print
(a,b,c) # 20 20 30
```

Problem 0 Points (20)

- Challenge 1: Two functions, maximum and minimum, return the larger and smaller of two numbers, respectively (6 points)
- Challenge 2: Valid date function returns True or False depending on whether or not the date is valid – tests 3 cases (9 points; 3 points each)
 - Test case 1: valid month, valid day -> Output = True
 - Test case 2: valid month, invalid day -> Output = False
 - Test case 3: invalid month, valid day -> Output = False
- Challenge 3: Simple sort function returns 3 numbers from smallest to largest (5 points)

Problem 1

You will be writing a series of programs that build on one another, so it's important to progress through this part in order. You only need to submit the final version of your code (no need to submit the intermediate steps that you took in order to reach the final program). When you're finished you should submit your program to the Assignment #6 category inside of Brightspace.

Part 1a: Number Analyzer

Write a series of five functions that determine if a given positive integer is EVEN, ODD, PRIME, PERFECT and ABUNDANT. Each of these functions should accept a single integer as an argument and return whether the integer meets the criteria for that classification. Here's some IPO notation to get you started:

```
# function: is_even
# input:    a positive integer
# processing: determines if the supplied number is even
# output:   Boolean

# function: is_odd
# input:    a positive integer
# processing: determines if the supplied number is odd
# output:   Boolean

# function: is_prime
# input:    a positive integer
# processing: determines if the supplied number is prime. a prime number is
#             a number that only has two factors - the number 1 and itself.
#             for example, 17 is prime because it only has two factors (1 and
#             17). in order to determine this you might need to count # of
```

```

#           factors between 1 and the number you are testing. a note on
#           efficiency: if you are testing a number (say, 15) and you find a
#           factor (say, 3) - do you need to even continue
#           to find additional factors?
# output:   Boolean

# function: is_perfect
# input:    a positive integer
# processing: determines if the supplied number is perfect. a perfect number
#           is a number that is equal to the sum of its factors (i.e. the
#           number 6 is perfect because  $6 = 1 + 2 + 3$ )
# output:   Boolean

# function: is_abundant
# input:    a positive integer
# processing: determines if the supplied number is abundant. an abundant #
#           is a number that is less than the sum of its factors (i.e. the
#           number 12 is abundant because  $12 < 1 + 2 + 3 + 4 + 6$ )
#           note: this includes proper divisors only (12 itself is excluded)
# output:   Boolean

```

Test your code before continuing!!! Make sure that your functions work as desired and then move onto the next step. Here's some sample code that will help you identify whether your functions are working correctly:

```

a1 = is_even(5)
a2 = is_even(6)
print (a1, a2) # False True

b1 = is_odd(5)
b2 = is_odd(6)
print (b1, b2) # True False

c1 = is_prime(5)
c2 = is_prime(17)

```

```
c3 = is_prime(21)
print (c1,c2,c3) # True True False

d1 = is_perfect(6)
d2 = is_perfect(7)
d3 = is_perfect(10)
print (d1,d2,d3) # True False False

e1 = is_abundant(12)
e2 = is_abundant(13)
e3 = is_abundant(14)
print (e1,e2,e3) # True False False
```

Once you're happy with your functions you should write a short program to prompt the user for a starting and ending number (both integers) - ensure that the first number is positive, and that the second number is greater than the first number. Then use your functions to analyze all numbers in this range to and generate output similar to the following:

```
Enter starting number: -5
Invalid, try again
Enter starting number: 5
Enter ending number: 0
Invalid, try again
Enter ending number: 30
5 is ... odd prime
6 is ... even perfect
7 is ... odd prime
8 is ... even
9 is ... odd
10 is ... even
11 is ... odd prime
12 is ... even abundant
13 is ... odd prime
14 is ... even
15 is ... odd
```

16 is ... even
17 is ... odd prime
18 is ... even abundant
19 is ... odd prime
20 is ... even abundant
21 is ... odd
22 is ... even
23 is ... odd prime
24 is ... even abundant
25 is ... odd
26 is ... even
27 is ... odd
28 is ... even perfect
29 is ... odd prime
30 is ... even abundant

Problem 1b: Number Analyzer

Next, write a program that prompts the allows the user to analyze numbers within a given range for numbers that fit the above criteria. The program should continue to execute as long as the user wishes to keep going. Your program should rely on the functions you wrote for problem 2a - you shouldn't have to re-write the routines that you worked out to compute whether a number is prime, abundant, perfect, odd or even. Here's a sample running of the program:

Main Menu

- 1 - Find all prime numbers within a given range
- 2 - Find all perfect numbers within a given range
- 3 - Find all abundant numbers within a given range
- 4 - Chart all even, odd, prime, perfect and abundant numbers within a given range
- 5 - Quit

Your choice: apple

I don't understand that ...

Main Menu

- 1 - Find all prime numbers within a given range
- 2 - Find all perfect numbers within a given range
- 3 - Find all abundant numbers within a given range
- 4 - Quit

Your choice: 1

Enter starting number (positive only): -5

Invalid, try again

Enter starting number (positive only): 5

Enter ending number: 3

Invalid, try again

Enter ending number: 20

All prime numbers between 5 and 20

#####

5

7

11

13

17

19

#####

Main Menu

- 1 - Find all prime numbers within a given range
- 2 - Find all perfect numbers within a given range
- 3 - Find all abundant numbers within a given range
- 4 - Quit

Your choice: 2

Enter starting number (positive only): 1

Enter ending number: 1000

All perfect numbers between 1 and 1000

#####

6

28

496

#####

Main Menu

- 1 - Find all prime numbers within a given range
- 2 - Find all perfect numbers within a given range
- 3 - Find all abundant numbers within a given range
- 4 - Quit

Your choice: 3

Enter starting number (positive only): 1

Enter ending number: 100

All abundant numbers between 1 and 100

#####

12

18

20

24

30

36

40
42
48
54
56
60
66
70
72
78
80
84
88
90
96
100

#####

Main Menu

- 1 - Find all prime numbers within a given range
- 2 - Find all perfect numbers within a given range
- 3 - Find all abundant numbers within a given range
- 4 - Quit

Your choice: 4

Goodbye!

Note that you might want to write some additional "helper" functions - for example, the options above all require the user to supply a range of numbers to analyze. You can probably write a function that does this so you don't have to continually copy and paste the same code over and over again.

Problem 1 Points (28)

- Negative numbers not allowed (2 points)

- Loop ensures highest number is larger than lowest number (2 points)
- 3 test cases – one for each of menu option 1-3, where correct output numbers are printed based on input range (6 points each)
- If 4 is entered, the program stops (3 points)
- Headings are printed for menu options 1, 2, and 3 (e.x. “All prime numbers between 4 and 40”) (3 points)

Problem 2

For part 2 of this assignment you will be writing a series of programs that build on one another, so it's important to progress through this part in order. You only need to submit the final version of your code (no need to submit the intermediate steps that you took in order to reach the final program) as well as the "module" that you create for Part 2e. When you're finished you should submit your program to the Assignment #6 category inside of Brightspace.

Problem 2a

For this step you are given three functions - go ahead and copy these functions into a new file called "digitalclock.py":

```
# function: horizontal_line
# input:    a width value (integer) and a single character (string)
# processing: generates a single horizontal line of the desired size
# output:   returns the generated pattern (string)
def horizontal_line(width,char):
    return width*char + "\n"

# function: vertical_line
# input:    a shift value and a height value (both integers) and a single character (string)
# processing: generates a single vertical line of the desired height. the line is
#            offset from the left side of the screen using the shift value
# output:   returns the generated pattern (string)
def vertical_line(shift,height,char):
    pattern = ""
    for i in range(height):
        pattern += shift*" " + char + "\n"
    return pattern
```

```

# function: two_vertical_lines
# input:    a width value and a height value (both integers) and a single character (string)
# processing: generates two vertical lines. the first line is along the left side of
#           the screen. the second line is offset using the "width" value supplied
# output:   returns the generated pattern (string)
def two_vertical_lines(width,height,char):
    pattern = ""
    for i in range(height):
        pattern += char + " "*(width-2) + char + "\n"
    return pattern

```

Next, create a file called 'LastName_FirstName_Assign6_part2.py' - make sure this file is in the same folder as your newly created 'digitalclock.py' file. Import your module and run the following code - you should be able to see a series of graphical patterns that match the output below:

```

import digitalclock

print ("Horizontal line, width = 5:")
temp = digitalclock.horizontal_line(5, '*')
print (temp)
print ()

print ("Horizontal line, width = 10:")
temp = digitalclock.horizontal_line(10, '+')
print (temp)
print ()

print ("Horizontal line, width = 15:")
temp = digitalclock.horizontal_line(15, 'z')
print (temp)
print ()

print ("Vertical Line, shift=0; height=3:")
temp = digitalclock.vertical_line(0, 3, '!')
print (temp)

```

```

print ()

print ("Vertical Line, shift=3; height=3:")
temp = digitalclock.vertical_line(3, 3, '&')
print (temp)
print ()

print ("Vertical Line, shift=6; height=5:")
temp = digitalclock.vertical_line(6, 5, '$')
print (temp)
print ()

print ("Two Vertical Lines, width=3; height=3:")
temp = digitalclock.two_vertical_lines(3, 3, '^')
print (temp)
print ()

print ("Two Vertical Lines, width=4; height=5:")
temp = digitalclock.two_vertical_lines(4, 5, '@')
print (temp)
print ()

print ("Two Vertical Lines, width=5; height=2:")
temp = digitalclock.two_vertical_lines(5, 2, '#')
print (temp)
print ()

```

Expected Output:

Horizontal line, width = 5:

```
*****
```

Horizontal line, width = 10:

```
++++++++++
```

Horizontal line, width = 15:

zzzzzzzzzzzzzzzz

Vertical Line, shift=0; height=3:

!

!

!

Vertical Line, shift=3; height=3:

&

&

&

Vertical Line, shift=6; height=5:

\$

\$

\$

\$

\$

Two Vertical Lines, width=3; height=3:

^ ^

^ ^

^ ^

Two Vertical Lines, width=4; height=5:

@ @

@ @

```
@ @  
@ @  
@ @
```

Two Vertical Lines, width=5; height=2:

```
# #  
# #
```

Problem 2b

As you can see, you have three "primitive" functions for generating simple shapes (horizontal lines, vertical lines and parallel vertical lines). Your next task is to write 10 new functions that generate the digits 0-9 using your three line functions. These functions should be stored in your 'digitalclock.py' module. The goal here is to render the digits as they would appear on a digital display:



Each function should accept a "width" argument to control how wide the number should be as well as a single character. You can assume numbers will always be printed with a height of 5. For example, here is the function for the number 1:

```
# function:  number_1  
# input:    a width value (integer) and a single character (string)  
# processing: generates the number 1 as it would appear on a digital display  
#           using the supplied width value  
# output:   returns the generated pattern (string)  
def number_1(width, character):  
    pattern = vertical_line(width-1, 5, character)  
    return pattern
```

And here's a sample program that calls the function a few times (test this in your main program, not in your module):

```
print ("Number 1, width=5: ")
```

```
temp = digitalclock.number_1(5, '*')
print(temp)
print()

print ("Number 1, width=10: ")
temp = digitalclock.number_1(10, '*')
print(temp)
print()

print ("Number 1, width=2: ")
temp = digitalclock.number_1(2, '*')
print(temp)
print()
```

And here's the expected output:

Number 1, width=5:

```
*
*
*
*
*
```

Number 1, width=10:

```
*
*
*
*
*
```

Number 1, width=2:

```
*
```

```
*  
  
*  
  
*  
  
*
```

Here's a sample program that prints all of the numbers (0-9).

```
temp = digitalclock.number_0(5, '**')  
print(temp)  
print()  
  
temp = digitalclock.number_1(5, '**')  
print(temp)  
print()  
  
temp = digitalclock.number_2(5, '**')  
print(temp)  
print()  
  
temp = digitalclock.number_3(5, '**')  
print(temp)  
print()  
  
temp = digitalclock.number_4(5, '**')  
print(temp)  
print()  
  
temp = digitalclock.number_5(5, '**')  
print(temp)  
print()  
  
temp = digitalclock.number_6(5, '**')
```



```
print(temp)
print()

temp = digitalclock.number_7(5, '**')
print(temp)
print()

temp = digitalclock.number_8(5, '**')
print(temp)
print()

temp = digitalclock.number_9(5, '**')
print(temp)
print()
```

And here's the expected output:

```
*****
*   *
*   *
*   *
*****

*
*
*
*
*

*****
*
```

*

*

*

* *

* *

*

*

*

*

*

* *

```

    *

    *

    *

    *

*****

*   *

*****

*   *

*****

*****

*   *

*****

    *

    *

```

Problem 2c

Write a function called 'print_number' that prints out any desired number to the screen. This function should also be placed in your 'digitalclock.py' module. Here's the IPO for this function:

```

# function:  print_number

# input:    a number to print (integer), a width value (integer) and a single character (string)

# processing: prints the desired number to the screen using the supplied width value

# output:   does not return anything

```

And here's some sample code that you can use to test your function:

```

digitalclock.print_number(0, 5, '*')
digitalclock.print_number(1, 6, '*')
digitalclock.print_number(2, 7, '*')
digitalclock.print_number(3, 8, '*')
digitalclock.print_number(4, 9, '*')
digitalclock.print_number(5, 10, '*')

```

```
digitalclock.print_number(6, 11, '**')
digitalclock.print_number(7, 12, '**')
digitalclock.print_number(8, 13, '**')
digitalclock.print_number(9, 14, '**')
```

And here's the expected output:

```
*****
*      *
*      *
*      *
*      *
*****

      *
      *
      *
      *
      *

*****
      *
*****
*
*****

*****
      *
*****
      *
*****

*      *
```

* *

 *

 *

*

 *

*

* *

 *

 *

 *

 *

* *

* *

```
*           *
*****
           *
           *
```

Problem 2d

Write two new functions that simulate the addition and subtraction operators. Each of these functions should accept a width value as an argument (integer) and a single character (string) -- the function should then return the generated pattern. You can assume the operators will always be 5 units high. Again, these functions should be placed in your 'digitalclock.py' module. Here's some sample code:

```
temp = digitalclock.plus(5, '*')
print(temp)
print()

temp = digitalclock.minus(5, '*')
print(temp)
```

Which will generate ...

```
*
*
*****
*
*

*****
```

Note that your "plus" sign may look odd if it is rendered using an even size value - for example:

```
# rendered using a width of 6
*
*
```

```
*****
```

```
 *
```

```
 *
```

To fix this you should double up the vertical line in the center for even sizes, like this:

```
# rendered using a width of 6
```

```
 **
```

```
 **
```

```
*****
```

```
 **
```

```
 **
```

Problem 2e

Write a function called "check_answer" which will determine if a given addition or subtraction problem was solved correctly. This function should be inside of your "digitalclock.py" module. Here's the IPO notation for the function:

```
# function:  check_answer
# input:    two numbers (number1 & number2, both integers);
#           an answer (an integer)
#           and an operator (+ or -, expressed as a String)
# processing: determines if the supplied expression is correct.  for example,
#           if the operator is "+", number1 = 1, number2 = 2 and answer = 3
#           then the expression is correct
#           (1 + 2 = 3).
# output:   returns True if the expression is correct, False if it is
#           not correct
```

Here's a sample program that you can use to test your function:

```
answer1 = digitalclock.check_answer(1, 2, 3, "+")
print (answer1)
answer2 = digitalclock.check_answer(1, 2, -1, "-")
print (answer2)
answer3 = digitalclock.check_answer(9, 5, 3, "+")
```

```
print (answer3)

answer4 = digitalclock.check_answer(8, 2, 4, "-")

print (answer4)
```

And here's the expected output:

```
True
True
False
False
```

Problem 2f

Finally, put everything together and write a program that lets the user practice a series of random addition and subtraction problems. Begin by asking the user for a number of problems (only accept positive values) and a size for their numbers (only accept numbers between 5 and 10). Also prompt them for a single character to be used to generate their patterns - only accept single character strings (i.e. 'a' is OK, but 'apple' is not). The generate a series of random addition and subtraction problems - display the numbers to the user with your digital display functions. Then prompt the user for an answer and check the answer using your check_answer function. Your program should also keep track of how many correct questions the user answered during their game. Here's a sample running of the program (user input is underlined):

```
How many problems would you like to attempt? -5
Invalid number, try again
How many problems would you like to attempt? 5
How wide do you want your digits to be? 5-10: 3
Invalid width, try again
How wide do you want your digits to be? 5-10: 5
What character would you like to use? foo
String too long, try again
What character would you like to use? *
Here we go!

What is .....

*****
```


*

*

*

*

*

*

*

*

*

*

*

= 4

Correct!

What is

*

*

*

*

*

*

= -5

Correct!

What is

*

*

*

*

*

*

*

= 0

Sorry, that's not correct.

What is

*

*

*

*

*

*

*

*

*

*

*

= 3

Correct!

What is

*

```
*  
*****
```

```
  *  
  *  
*****  
  *  
  *
```

```
*****  
  *  
*****  
  *  
*****
```

= 4

Correct!

You got 4 out of 5 correct!

Problem 2 Points (52)

- Digital clock module is imported (3 points) – see 2a for explanation
- 10 functions are created, one for each digit 0-9, which take 2 arguments, a width and a character (5 points, .5 point each) – see 2b for explanation
- Print_number function is created, takes a number, a width, and a character as arguments (5 points) – see 2c for explanation
- Plus function prints plus sign – odd width prints single vertical line (2 points) and even width prints double vertical line (2 points) – see 2d for explanation
- Minus function prints minus sign (2 points) – see 2d for explanation
- Check_answer function passes 4 cases (2 points each) – see 2e for example
- 2f: # of problems attempted only accepts a positive number (3 points)

- 2f: width only accepts a number between 5 and 10 (3 points)
- 2f: Only a single character can be used to print output (3 points)
- 2f: Program executes the correct number of problems based on number input by user (3 points)
- 2f: Functions are used to print out problems as digital numbers (4 points)
- 2f: check_answer is used to print "Correct" if user enters correct answer and "Sorry, that's not correct" otherwise (4 points)
- 2f: Problems (numbers and addition/subtraction) are randomly selected (5 points)